

Study Case Fullstack Engineer

Tujuan: Kami ingin melihat kemampuanmu membangun sebuah aplikasi fullstack dari nol sampai deployable (Docker + Nginx).

0. Overview

- **Yang dibangun:** Aplikasi mini fullstack bernama **Wrapped Weekly** (detail di bagian 2).
- **Stack wajib:** Backend **Go + Gin + PostgreSQL**, Frontend **Next.js + shadcn/ui**.
- **Deliverable inti:** Aplikasi jalan lokal via satu perintah docker compose up, lengkap dengan **Dockerfile** untuk BE & FE dan **konfig Nginx** sebagai reverse proxy.
- **Nilai tambah (bonus):** Benar-benar men-*deploy* ke sebuah VPS/cloud dan kasih URL yang bisa diakses.
- **Waktu:** kurang lebih 3–5 hari (effort $\pm$ 12–16 jam). Jangan overengineering.
- **AI boleh di-mock.** Kami menilai arsitektur fullstack-mu.

1. Yang kami nilai

Dimensi	Bobot	Apa yang kami lihat
Backend (Go)	25%	Struktur berlapis yang rapi, validasi input, error handling, migration, query yang benar
Frontend (Next.js)	25%	Struktur kode, data fetching, UX, state & form (validasi), loading/empty/error states
Kualitas domain logic	15%	Agregasi mingguan & statistik benar, konsisten, dan tahan edge case
Dockerization + Nginx	20%	docker compose up jalan sekali klik, Nginx reverse proxy benar
Kerapian & dokumentasi	15%	README jelas, commit history rapi, .env.example lengkap
Deploy sungguhan (BONUS)	+15%	URL live yang bisa kami akses

Dimensi	Bobot	Apa yang kami lihat
		end-to-end

Total 100% + bonus. **Deployment beneran = pembeda kandidat kuat**, tapi bukan syarat lulus.

2. Brief Aplikasi "Wrapped Weekly"

Pitch: Aplikasi yang mengubah aktivitas mingguanmu menjadi *recap* ala "Spotify Wrapped". User mencatat aktivitas sepanjang minggu (mis. olahraga, baca buku, ngoding, pengeluaran), lalu tiap minggu app menghasilkan **recap otomatis**: statistik + narasi bergaya AI + kartu yang bisa **dibagikan lewat link publik**.

Kenapa app ini? Ukurannya pas untuk beberapa hari, tapi menyentuh semua otot fullstack: auth, CRUD, **agregasi/statistik non-trivial**, integrasi API eksternal (LLM), **halaman publik SSR yang shareable**, dan dashboard visual.

2.1 Fitur / Must Have (MVP)

- Auth**
 - Register & Login (email + password). Password wajib di-hash.
 - JWT access token (refresh token = nilai plus). Endpoint privat butuh Bearer token.
- Activity Logging (CRUD)**
 - User mencatat aktivitas: category (mis. workout / reading / coding / spending), value (angka — menit, halaman, rupiah, dsb.), note?, dan occurred_at.
 - Bisa lihat / edit / hapus entri milik sendiri saja (otorisasi per-user).
- Weekly Recap (fitur inti)**
 - Untuk satu rentang minggu (Senin–Minggu), agregasikan aktivitas user menjadi statistik: total per kategori, kategori terbanyak, hari paling produktif, perubahan vs minggu sebelumnya, dsb.
 - Hasilkan **narasi recap** bergaya "Wrapped" berdasarkan statistik itu (dikirim ke LLM **atau** di-*mock* deterministik jika tanpa API key).
 - Recap bisa dibuat/di-*generate* dan disimpan.
- Dashboard**
 - Ringkasan minggu berjalan + minimal **1 chart** (mis. aktivitas per hari / komposisi kategori). Boleh pakai library chart apa pun (mis. Recharts).
- Shareable Public Recap**
 - Tiap recap punya **URL publik** (mis. /w/{slug}) yang bisa diakses **tanpa login** dan menampilkan kartu recap-nya.
 - Halaman ini harus **SSR** dan punya **meta tag Open Graph** yang benar (title/description/image) supaya link preview-nya bagus saat dibagikan.

2.2 Nice to Have (bonus, kerjakan hanya jika sempat)

- **Scheduled job / cron** yang otomatis meng-*generate* recap tiap awal minggu untuk semua user.
- **Integrasi API eksternal** sebagai sumber aktivitas otomatis (mis. GitHub commits, Strava, atau RSS) menggantikan input manual.
- **Streaming** narasi AI ke UI.
- **OG image dinamis** untuk kartu share.
- **Redis** untuk cache recap publik atau rate-limit endpoint AI.
- **Pagination & filter** pada riwayat aktivitas.
- **Dark mode** + empty/loading/error states yang rapi.
- **Unit test** untuk fungsi agregasi/statistik (pembeda kuat logic ini rawan bug).

Jangan kejar semua nice-to-have. Lebih baik MVP solid + 1–2 bonus yang matang.

3. Aturan Teknis

Backend Go

- **Go + Gin**, disusun berlapis (mis. domain / usecase / repository / handler) dengan pemisahan tanggung jawab yang jelas.
- **PostgreSQL** sebagai database. ORM bebas (mis. GORM) atau query manual.
- Skema dikelola via **migration SQL** (mis. golang-migrate), bukan auto-migrate saat runtime.
- Route di bawah prefix **/api/v1**, plus endpoint **/health** untuk healthcheck.
- **JWT** untuk auth; lindungi endpoint privat dengan middleware.
- Config lewat environment variable (.env + .env.example).
- Format response konsisten (mis. envelope { "data": ..., "message": ... }).

Frontend Next.js

- **Next.js (App Router) + TypeScript**, output **standalone** (memudahkan Docker).
- **shadcn/ui + Tailwind** untuk UI.
- Data fetching yang rapi (mis. TanStack Query) + form dengan validasi (mis. React Hook Form + Zod).
- Susun kode dengan rapi (mis. per-fitur), endpoint API terpusat di satu tempat.
- Base URL API dari env NEXT_PUBLIC_API_URL.
- Halaman recap publik memakai **Server-Side Rendering** + metadata Open Graph.

4. Deliverables

Submit dalam **satu repo Git** (boleh monorepo dengan folder backend/ & frontend/, atau 2 repo terpisah, jelaskan di README) berisi:

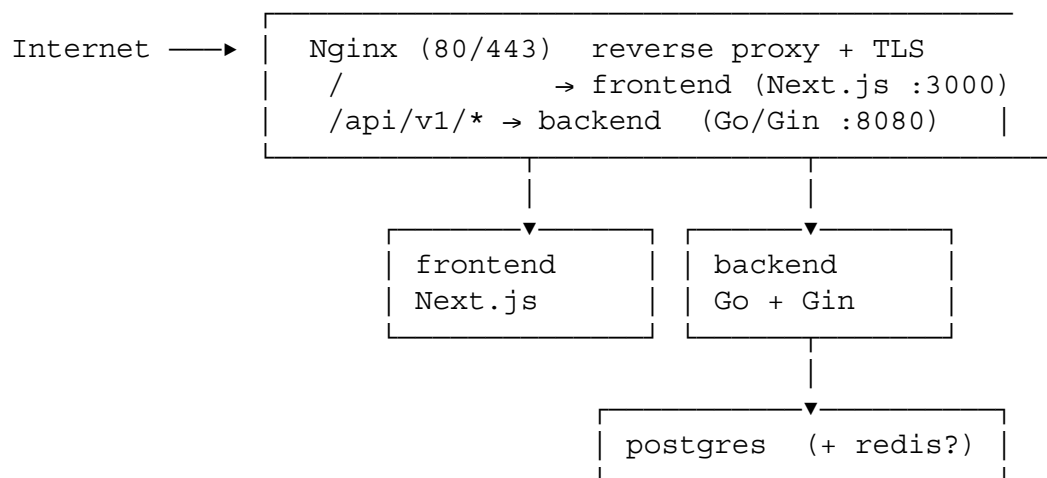
1. **Source code** BE + FE.
2. **README.md**, cara run lokal, arsitektur singkat, **rumus/aturan agregasi recap**, daftar fitur selesai vs belum, dan keputusan teknis & trade-off.

3. **Dockerfile** untuk BE dan FE.
4. **docker-compose.yml** yang menyalakan semuanya (db + backend + frontend + nginx).
5. **Konfig Nginx** (mis. di folder nginx/ atau `deploy/`).
6. **.env.example** untuk BE & FE (jangan commit secret asli!).
7. (Bonus) **URL live** + catatan cara deploy di README.

5. Panduan Deploy Step by Step

Ini fokus utama study case. Target: dari repo > **docker compose up jalan lokal** > (bonus) **live di server**.

5.1 Arsitektur target



5.2 Dockerfile Backend (Go, multi-stage) contoh acuan

Build binary statis lalu jalankan di image kecil sebagai non-root user.

```

# ---- build ----
FROM golang:1.25-alpine AS builder
WORKDIR /app
COPY go.mod go.sum ./
RUN go mod download
COPY . .
RUN CGO_ENABLED=0 GOOS=linux go build -o /out/server ./cmd/api

# ---- run ----
FROM alpine:3.20
RUN adduser -D -u 10001 appuser
WORKDIR /app
COPY --from=builder /out/server .
USER appuser
EXPOSE 8080
  
```

```
HEALTHCHECK --interval=30s --timeout=3s CMD wget -qO-  
http://localhost:8080/health || exit 1  
ENTRYPOINT ["/server"]
```

5.3 Dockerfile Frontend (Next.js standalone) contoh acuan

Pastikan next.config.ts memakai output: "standalone".

```
# ---- deps ----  
FROM node:22-alpine AS deps  
WORKDIR /app  
COPY package.json package-lock.json ./  
RUN npm ci  
  
# ---- build ----  
FROM node:22-alpine AS builder  
WORKDIR /app  
COPY --from=deps /app/node_modules ./node_modules  
COPY . .  
ARG NEXT_PUBLIC_API_URL  
ENV NEXT_PUBLIC_API_URL=$NEXT_PUBLIC_API_URL  
RUN npm run build  
  
# ---- run ----  
FROM node:22-alpine AS runner  
WORKDIR /app  
ENV NODE_ENV=production  
RUN addgroup -g 1001 nodejs && adduser -D -u 1001 -G nodejs nextjs  
COPY --from=builder /app/public ./public  
COPY --from=builder --chown=nextjs:nodejs /app/.next/standalone ./  
COPY --from=builder --chown=nextjs:nodejs /app/.next/static  
./ .next/static  
USER nextjs  
EXPOSE 3000  
CMD ["node", "server.js"]
```

Catatan: NEXT_PUBLIC_* di-bake saat build. Kalau frontend memanggil backend dari browser, arahkan lewat Nginx (/api/v1) supaya bebas masalah CORS/URL internal.

5.4 docker-compose.yml contoh acuan

```
services:  
  postgres:  
    image: postgres:16-alpine  
    environment:  
      POSTGRES_USER: ${DB_USER}  
      POSTGRES_PASSWORD: ${DB_PASSWORD}  
      POSTGRES_DB: ${DB_NAME}
```

```

volumes: [pgdata:/var/lib/postgresql/data]
healthcheck:
  test: ["CMD-SHELL", "pg_isready -U ${DB_USER}"]
  interval: 5s
  timeout: 5s
  retries: 5

migrate:
  image: migrate/migrate
  depends_on:
    postgres: { condition: service_healthy }
  volumes: ["/backend/db/migrations:/migrations"]
  command:
    ["-path", "/migrations", "-database",

"postgres://${DB_USER}:${DB_PASSWORD}@postgres:5432/${DB_NAME}?sslmode
=disable",
    "up"]

backend:
  build: ./backend
  env_file: ./backend/.env
  depends_on:
    postgres: { condition: service_healthy }
    migrate: { condition: service_completed_successfully }
  expose: ["8080"]

frontend:
  build:
    context: ./frontend
  args:
    NEXT_PUBLIC_API_URL: ${NEXT_PUBLIC_API_URL}
  depends_on: [backend]
  expose: ["3000"]

nginx:
  image: nginx:alpine
  depends_on: [frontend, backend]
  ports: ["80:80"]          # tambahkan "443:443" saat pakai TLS
  volumes:
    - ./nginx/app.conf:/etc/nginx/conf.d/default.conf:ro

volumes:
  pgdata:

```

migrate sengaja dipisah sebagai service sekali-jalan: ia menjalankan migration sampai selesai sebelum backend boleh start (service_completed_successfully), jadi skema DB selalu siap sebelum API menyala.

5.5 Konfig Nginx contoh acuan (nginx/app.conf)

```
upstream frontend { server frontend:3000; }
upstream backend { server backend:8080; }

server {
    listen 80;
    server_name _;

    # API → backend
    location /api/v1/ {
        proxy_pass http://backend;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }

    # sisanya → frontend
    location / {
        proxy_pass http://frontend;
        proxy_set_header Host $host;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
```

5.6 Menjalankan (lokal)

```
# 1. Siapkan env
cp backend/.env.example backend/.env
cp .env.example .env          # DB_USER, DB_PASSWORD, DB_NAME,
NEXT_PUBLIC_API_URL, dst

# 2. Build & nyalakan semua
docker compose up --build -d

# 3. Cek health backend
curl http://localhost/api/v1/health # atau /health sesuai route-mu

# 4. Buka app
open http://localhost
```

5.7 Deploy ke server (BONUS) outline

1. Siapkan VPS (mis. DigitalOcean / GCP e2-small / AWS Lightsail), install Docker + Docker Compose.
2. Arahkan **domain/subdomain** ke IP server (A record).
3. git clone repo di server, isi .env produksi (secret asli jangan masuk repo).
4. docker compose up --build -d.
5. **TLS**: pasang sertifikat Letsencrypt (Certbot), lalu tambahkan blok listen 443 ssl, redirect 80 > 443, dan security headers (HSTS dll) di config Nginx.
6. Verifikasi alur end-to-end: register → catat aktivitas > generate recap > buka **link publik recap** (cek preview OG) → dashboard.
7. Tulis URL + langkah singkat di README.

Alternatif deploy yang juga kami terima: Frontend di Vercel + Backend & DB di Railway/Render/Fly.io. Yang penting **URL-nya hidup** dan alur end-to-end jalan. Jelaskan pilihanmu di README.

6. Tips & Batasan

- **Scope kecil, eksekusi rapi.** MVP solid + deploy jalan jauh lebih mengesankan daripada 10 fitur setengah jadi.
- **Commit history yang bercerita** lebih baik dari satu commit raksasa "final".
- **Mock AI itu wajar** — sediakan flag AI PROVIDER = mock yang mengembalikan narasi deterministik agar app tetap jalan tanpa API key. Jelaskan cara mengaktifkan provider asli di README.
- **Jangan commit secret.** Gunakan .env.example.
- **Edge case agregasi** adalah tempat kami mengintip ketelitianmu: timezone, batas minggu (Senin–Minggu), minggu tanpa aktivitas, pembagian saat data kosong.
- Kalau ada keputusan yang kamu ragu, **tulis asumsimu di README** itu justru bernilai plus.

7. Cara Submit

1. Push ke GitHub/GitLab (private, invite reviewer) **atau** kirim .zip repo lengkap dengan .git.
2. Pastikan README.md memuat: cara run lokal (docker compose up), arsitektur, aturan agregasi recap, fitur selesai vs belum, dan (bonus) URL live.
3. Sertakan estimasi waktu yang kamu habiskan.